PNY Trainings — Cyber Security Department

Web Application Security — Final Project Assignments

Project 4: Harden and Rescan

Security-Header Hardening of OWASP Juice Shop
Baseline Scan, Remediation & Rescan Comparison

Student: Hamza Iqbal

Environment: Local lab — Docker on Windows 11 (own host)

Table of Contents

1. Objective

2. Scope & Approach

3. Tools Used

4. Baseline Scan Results (Before)

5. Manual Verification of the Targeted Findings

6. Hardening Applied — nginx Reverse Proxy

7. Verification of the Fixes

8. Rescan & Before/After Comparison

9. Residual Risk & Further Recommendations

10. Conclusion

1. Objective

The objective of this project was to take the target application from assessment to remediation: run a baseline security scan, manually verify at least three of the reported findings, apply real fixes for them, and then rescan the hardened application to produce an evidence-based before/after comparison.

2. Scope & Approach

- **Target —** OWASP Juice Shop v20.3.0 in Docker at http://localhost:3000 (baseline) — unchanged application.
- **Environment —** Isolated lab on the tester's own Windows 11 host; no external system was touched.
- **Approach —** The application image itself was left untouched. Hardening was applied at a reverse proxy (nginx) placed in front of the application, which then serves the site at http://localhost:4000. This mirrors a common production pattern — response-header policy is frequently enforced at the edge (proxy/WAF) rather than in application code — and it applies the headers uniformly to every response, including API responses the application itself leaves unprotected.

3. Tools Used

Tool	Version	Purpose
OWASP ZAP	2.17.0	Baseline scan and post-hardening rescan (Automated Scan: spider + passive/active, Dev Standard policy)
Docker Desktop	4.95.0	Runs the target application and the nginx hardening proxy
nginx (official Docker image)	latest (1.29.x)	Reverse proxy enforcing the security-header policy
Google Chrome + DevTools	155	Manual verification of response headers before and after
OWASP Juice Shop	20.3.0 (bkimminich/juice-shop)	Target application under test

4. Baseline Scan Results (Before)

A ZAP Automated Scan of http://localhost:3000 (traditional + client spider, Dev Standard policy) produced 8 alerts at the baseline capture point — 0 High, 4 Medium, 3 Low, 1 Informational:

#	Alert	Risk	Disposition (Section 8)
1	Content Security Policy (CSP) Header Not Set (Systemic)	Medium	Targeted — fixed
2	Cross-Domain Misconfiguration (Systemic)	Medium	Targeted — fixed
3	Missing Anti-clickjacking Header	Medium	Targeted — fixed
4	Session ID in URL Rewrite (Systemic)	Medium	Residual — application design
5	Private IP Disclosure	Low	Residual — response body content
6	Timestamp Disclosure — Unix (4)	Low	Residual — informational leakage
7	X-Content-Type-Options Header Missing (3)	Low	Targeted — fixed
8	Modern Web Application (Systemic)	Informational	Not a vulnerability

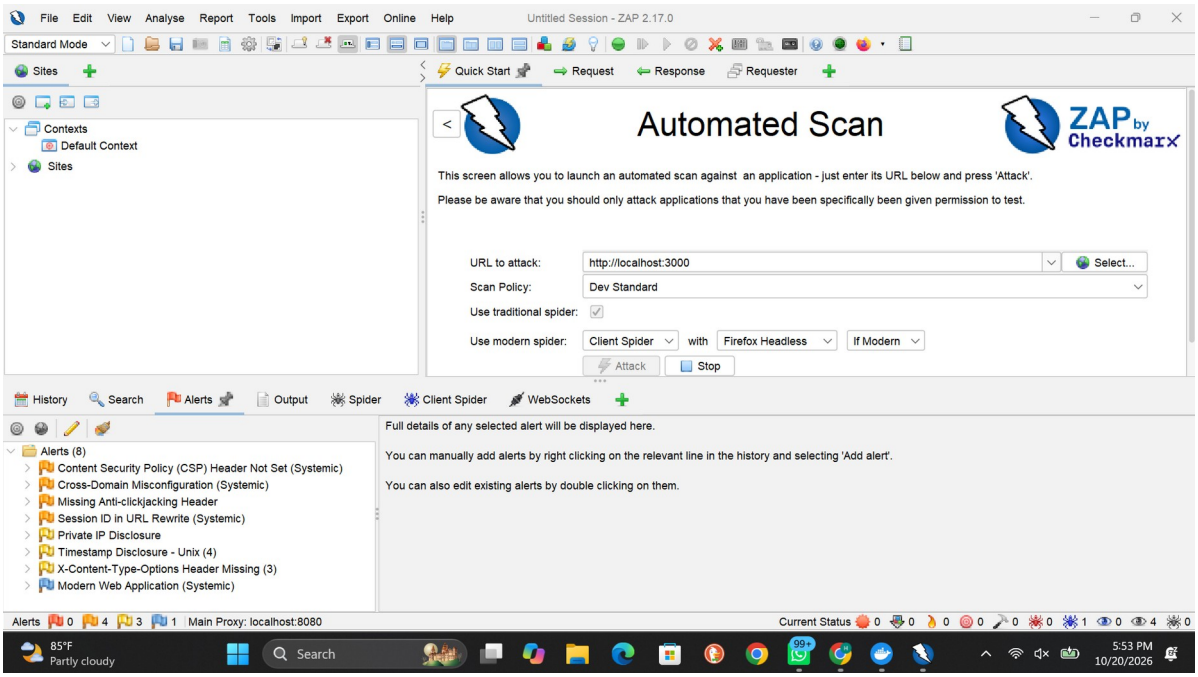


Figure 1 — Baseline: the 8 ZAP alerts against http://localhost:3000

Four alerts were selected for remediation: the three missing-header findings (CSP, anti-clickjacking, X-Content-Type-Options) and the cross-domain (CORS) misconfiguration — all four are response-header issues that a proxy can correct uniformly.

5. Manual Verification of the Targeted Findings

Each targeted finding was verified by hand in Chrome DevTools (Network → document request → Response Headers) before any fix was attempted:

5.1 Content-Security-Policy — absent (confirmed)

The response-header list contains no Content-Security-Policy header (Figure 2). Browsers therefore apply no content restrictions, leaving the application without its main defence-in-depth control against XSS impact (see Project 2).

5.2 CORS — wildcard origin (confirmed)

The same response carries Access-Control-Allow-Origin: * (Figure 2): any website may make cross-origin requests and read the responses — ZAP's "Cross-Domain Misconfiguration".

5.3 X-Frame-Options / X-Content-Type-Options — inconsistent (confirmed with nuance)

The main document does carry X-Frame-Options: SAMEORIGIN and X-Content-Type-Options: nosniff (Figure 3) — they are set by the static file server. ZAP nevertheless flagged both, because the application's API responses (which make up most of its traffic) do not include them: the protection exists on the shop window but not on the data behind it. A correct fix must therefore apply the headers to every response, not just to pages.

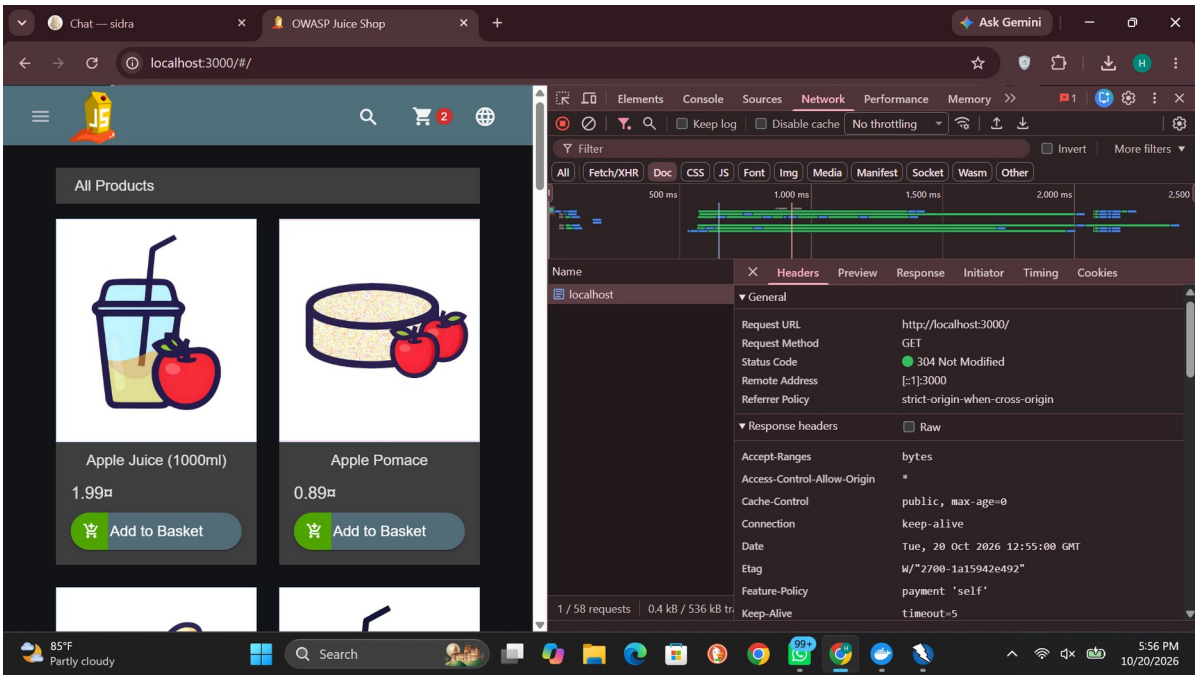


*Figure 2 — Before (document response, :3000): Access-Control-Allow-Origin: * and no Content-Security-Policy header*

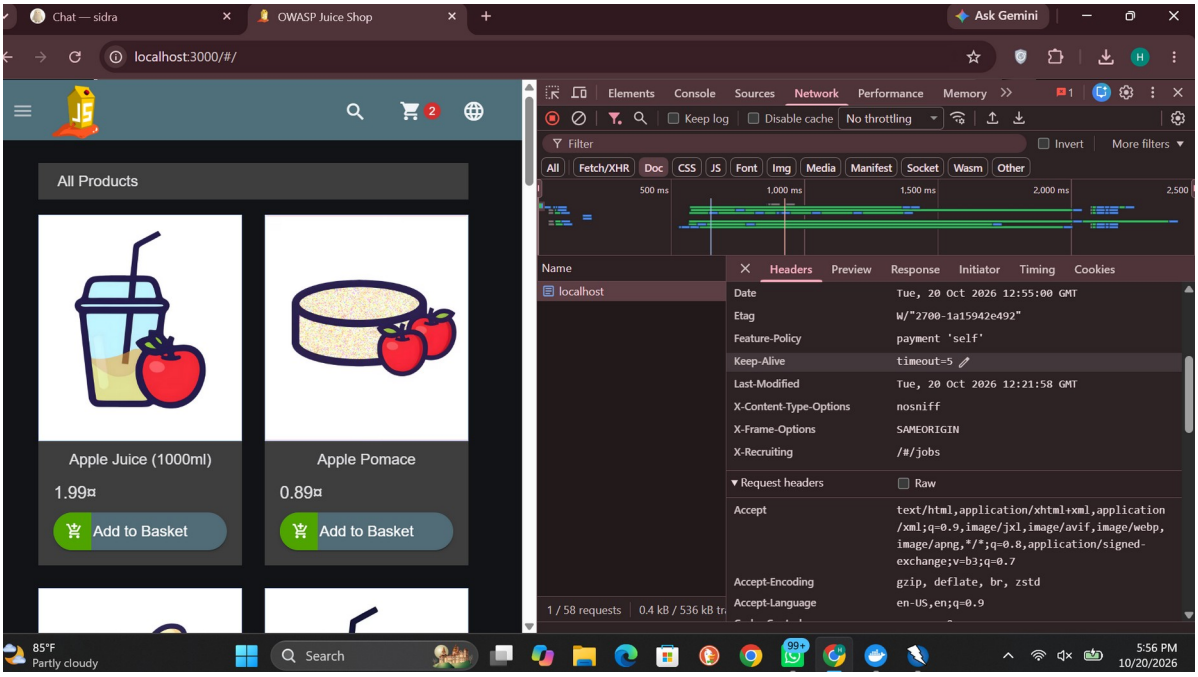


Figure 3 — Before (same response, scrolled): X-Frame-Options and X-Content-Type-Options present on the static document only

6. Hardening Applied — nginx Reverse Proxy

An nginx container (harden-proxy) was deployed in front of the application. Clients now reach the shop at http://localhost:4000; nginx forwards traffic to the unchanged application on port 3000 and rewrites the response headers on the way back. The deployed configuration:

```
map $http_upgrade $connection_upgrade { default upgrade; '' close; }

server {
    listen 80;
    server_tokens off;

    location / {
        proxy_pass http://host.docker.internal:3000;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection $connection_upgrade;

        proxy_hide_header X-Frame-Options;
        proxy_hide_header X-Content-Type-Options;
        proxy_hide_header Access-Control-Allow-Origin;

        add_header Content-Security-Policy "default-src 'self'; script-src 'self'
            'unsafe-inline' 'unsafe-eval'; style-src 'self' 'unsafe-inline';
            img-src 'self' data: blob:; font-src 'self' data:; connect-src 'self'
            ws: wss:; frame-src 'self' https://www.youtube.com;
```

```
            frame-ancestors 'none'; base-uri 'self'; form-action 'self'" always;
        add_header X-Frame-Options "SAMEORIGIN" always;
        add_header X-Content-Type-Options "nosniff" always;
        add_header Access-Control-Allow-Origin "http://localhost:4000" always;
        add_header Referrer-Policy "no-referrer" always;
    }
}
```

Design notes:

- **Uniformity —** proxy_hide_header + add_header guarantees exactly one instance of each managed header on every response — static, API, or error — with no duplicates.
- **Compatibility —** The CSP allows 'unsafe-inline'/'unsafe-eval' for scripts. Juice Shop's Angular frontend requires them to function; this is a deliberate, documented compatibility trade-off (discussed again in Section 8), not an oversight.
- **Clickjacking —** frame-ancestors 'none' in the CSP plus X-Frame-Options: SAMEORIGIN give clickjacking protection in modern and legacy browsers respectively.
- **Disclosure —** The CORS wildcard is replaced by the single legitimate origin; server_tokens off stops nginx leaking its version (the Server header now reads simply "nginx").

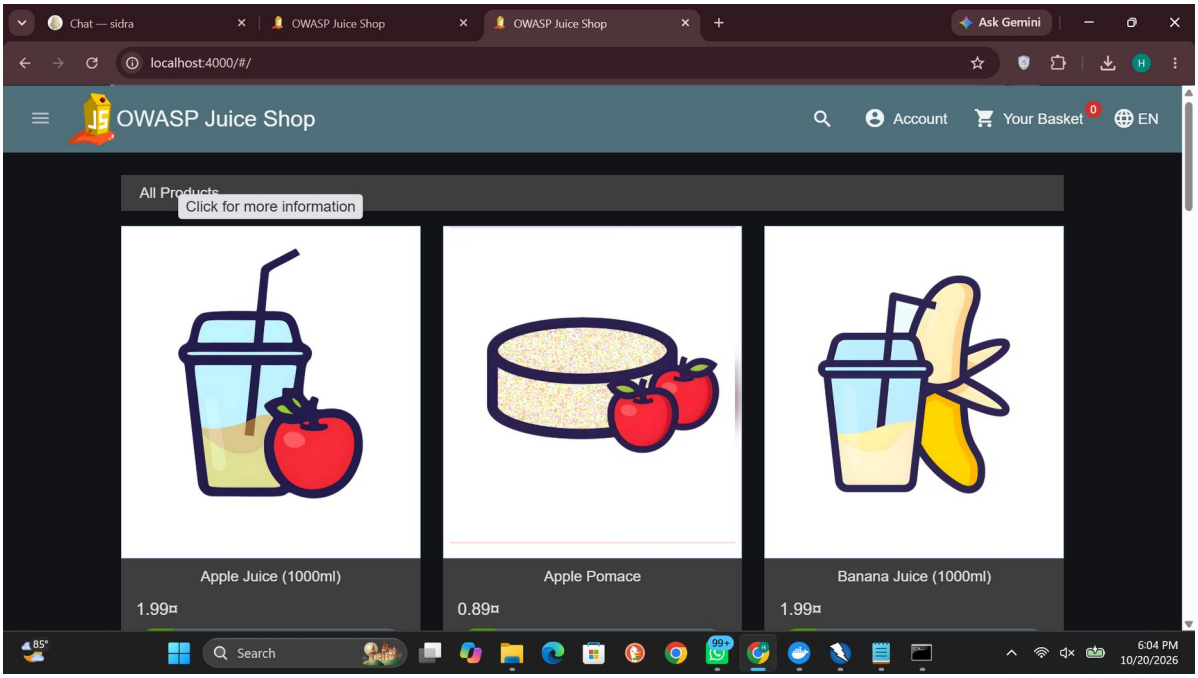


Figure 4 — The application running unchanged behind the hardening proxy at http://localhost:4000

7. Verification of the Fixes

Before rescanning, the fixes were verified by hand through the proxy (DevTools, document request to http://localhost:4000, Figure 5 and Figure 6):

- **CSP —** Content-Security-Policy: present, with the full policy from Section 6.

- **CORS —** Access-Control-Allow-Origin: http://localhost:4000 — the wildcard is gone.
- **Framing / sniffing —** X-Frame-Options: SAMEORIGIN and X-Content-Type-Options: nosniff — present exactly once each, now applied by the proxy to all responses.
- **Extras —** Referrer-Policy: no-referrer added; Server header reduced to "nginx" with no version.

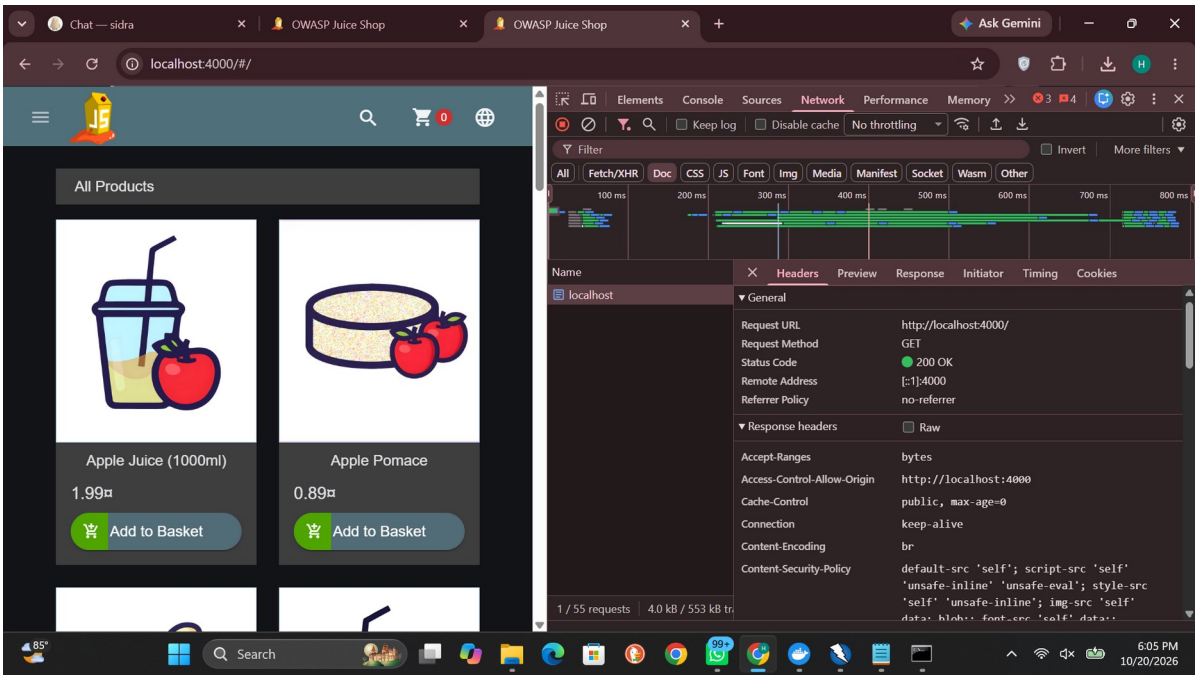


Figure 5 — After (document response via :4000): Content-Security-Policy present; Access-Control-Allow-Origin restricted to http://localhost:4000

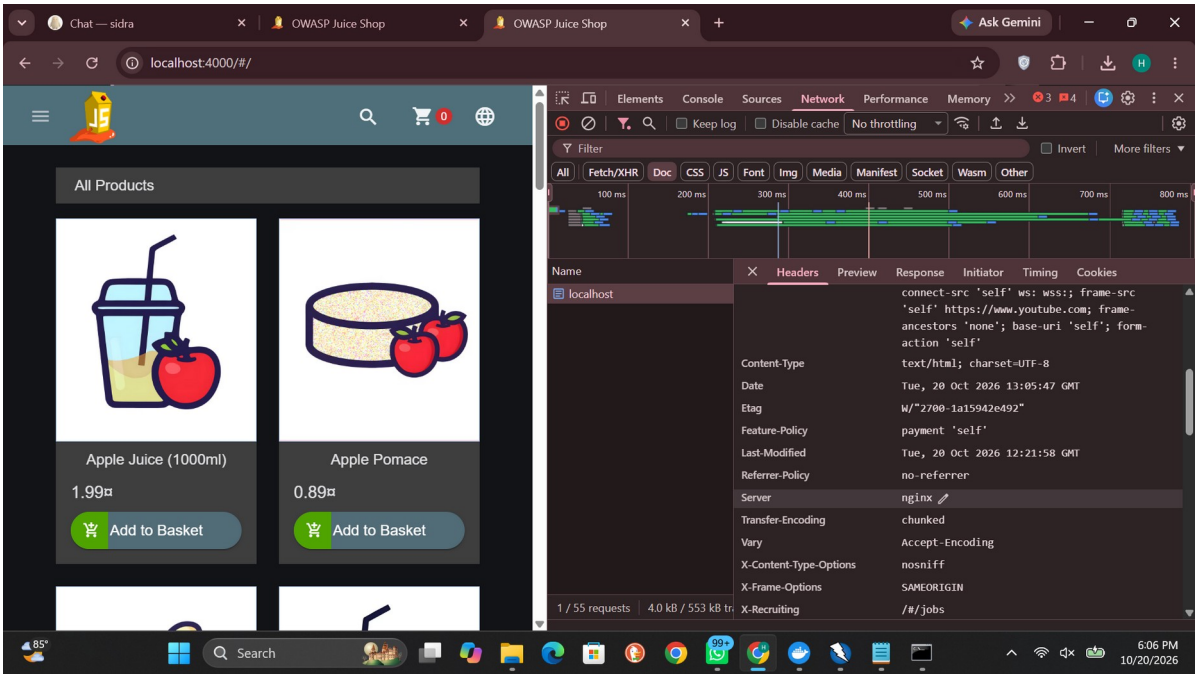


Figure 6 — After (same response, scrolled): single X-Frame-Options and X-Content-Type-Options headers; Server reduced to "nginx"

8. Rescan & Before/After Comparison

ZAP was relaunched in a fresh session and the identical Automated Scan (same policy, same spiders) was run against the hardened address, http://localhost:4000. Headline counts are superficially similar — 8 alerts before, 8 after — but the composition changed completely, which is why the comparison below is made alert by alert rather than by totals.

Baseline alert (:3000)	Risk	After rescan (:4000)	Verdict
CSP Header Not Set	Medium	Gone — replaced by CSP policy-quality notices (below)	RESOLVED
Cross-Domain Misconfiguration	Medium	Gone	RESOLVED
Missing Anti-clickjacking Header	Medium	Gone	RESOLVED
X-Content-Type-Options Missing	Low	Gone	RESOLVED
Session ID in URL Rewrite	Medium	Still present (socket.io session ids in URLs)	RESIDUAL
Private IP Disclosure	Low	Still present (internal addresses inside the app configuration body)	RESIDUAL
Timestamp Disclosure — Unix	Low	Still present	RESIDUAL
Modern Web Application	Info	Still present	NOT A FLAW

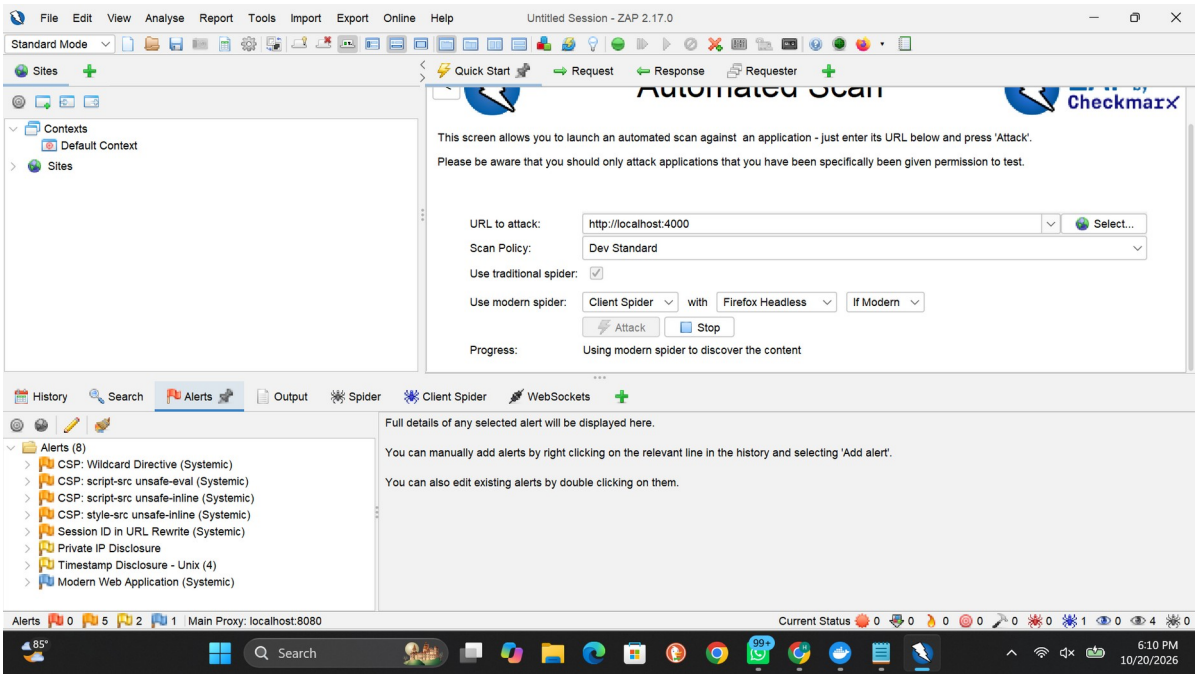


Figure 7 — Rescan: ZAP alerts against the hardened http://localhost:4000

New alerts introduced by the fix — and why they are progress

The rescan lists four CSP notices that did not exist at baseline: CSP Wildcard Directive, CSP script-src unsafe-eval, CSP script-src unsafe-inline, and CSP style-src unsafe-inline (all Medium). These are not regressions: at baseline there was no policy for ZAP to grade at all, and "no policy" was itself the Medium finding. The scanner now finds a policy and critiques its strictness. The critique is accurate — the policy deliberately permits inline/eval script for Angular compatibility (Section 6) — and it defines the next hardening step precisely: eliminate the application's reliance on inline script (nonce- or hash-based CSP) and on eval, then remove both keywords. The finding the project set out to fix — a missing CSP — is resolved; what remains is policy tuning, tracked as accepted risk with a documented path.

Result against the assignment criteria

- Findings verified manually before fixing: 4 (exceeds the required 3).
- Targeted findings resolved on rescan: 4 of 4.
- Application functionality through the proxy: unaffected (shop, basket, websockets all verified working).

9. Residual Risk & Further Recommendations

- **App change needed —** Session ID in URL Rewrite (Medium): the realtime channel places session identifiers in request URLs, where they leak via logs and Referer headers. Requires an application-level change (cookie/header-based session transport); cannot be corrected by response headers.
- **App change needed —** Private IP Disclosure (Low): internal addresses appear inside the application's configuration response — the same exposure reported as Finding 1 of Project 3. Fix at the source by removing internal data from the client-facing configuration.
- **Policy tuning —** CSP strictness: schedule the move to a nonce-based policy without 'unsafe-inline'/'unsafe-eval' (Section 8).
- **Deployment —** The proxy centralizes header policy; the same pattern should also terminate TLS and add Strict-Transport-Security before any real deployment.

10. Conclusion

Project 4 is complete. A ZAP baseline of the unmodified application produced 8 alerts; the four response-header findings among them (missing CSP, CORS wildcard, missing anti-clickjacking header, inconsistent X-Content-Type-Options) were each verified by hand, then fixed uniformly by placing an nginx reverse proxy in front of the application. Manual verification showed every intended header in place, and the ZAP rescan confirmed all four targeted findings resolved, with the application fully functional behind the proxy. The exercise also demonstrates the honest limits of edge hardening: the remaining findings live in the application's design and body content, and no proxy header can fix them — a distinction the before/after, alert-by-alert comparison makes visible.